

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра “Программная инженерия”

Отчет по Лабораторной работе №6.

Выполнил: Студент группы БПИ2404

Сидельников Николай Станиславович

Проверил: Адедиха Коффи Жермен

Москва

2026

Цель работы

Изучить основы тестирования Spring Boot-приложений и научиться использовать JUnit и Mockito для тестирования сервисов и контроллеров.

Теоретические сведения

JUnit - основной Java-фреймворк для тестирования.
Mockito - библиотека для создания mock-объектов.

Unit-тесты позволяют проверять отдельные классы изолированно.
Mock-объекты заменяют реальные зависимости приложения.

В Spring Boot чаще всего тестируются:

- service слой;
- controller слой;
- business logic;
- обработка исключений.

Структура тестов проекта

Тест	Назначение
SimpleTest	Базовый пример JUnit теста
UserServiceTest	Тестирование UserService
UserServiceVerifyTest	Проверка вызова repository.save()
NotificationServiceTest	Тестирование NotificationService
UserControllerTest	Web-тест контроллера

Разбор pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

<https://maven.apache.org/xsd/maven-4.0.0.xsd>">

```
        <modelVersion>4.0.0</modelVersion>
        <parent>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-parent</artifactId>
            <version>4.0.5</version>
            <relativePath/> <!-- lookup parent from repository -->
        </parent>
        <groupId>org.example</groupId>
        <artifactId>spring-lab3-notifications</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>spring-lab3-notifications</name>
        <description>spring-lab3-notifications</description>
        <url/>
        <licenses>
            <license/>
        </licenses>
        <developers>
            <developer/>
        </developers>
        <scm>
            <connection/>
            <developerConnection/>
            <tag/>
            <url/>
        </scm>
        <properties>
            <java.version>17</java.version>
```

```

</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>

  <!--      JPA      -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!--      PostgreSQL      -->
  <dependency>
    <groupId>org.postgresql</groupId>

```

```
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!--      Lombok      -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <!--      Validation      -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!--      Spring      Security      -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!--      JWT      -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
```

```

</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.12.6</version>
<scope>runtime</scope>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.12.6</version>
<scope>runtime</scope>
</dependency>

```

```

<!-- Tests: JUnit, Mockito, AssertJ, Spring Test -->
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-core</artifactId>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-junit-jupiter</artifactId>

```

```
<scope>test</scope>
```

```
</dependency>
```

```
</dependencies>
```

```
<build>
```

```
<plugins>
```

```
<plugin>
```

```
<groupId>org.springframework.boot</groupId>
```

Разбор SimpleTest

```
package org.example;
```

```
import org.junit.jupiter.api.Test;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
class SimpleTest {
```

```
    @Test
```

```
    void shouldAddNumbers() {
```

```
        int result = 2 + 3;
```

```
        assertEquals(5, result);
```

```
    }
```

```
    @Test
```

```
    void shouldCheckBasicAssertions() {
```

```

        String value = "Spring";

        assertNotNull(value);
        assertTrue(value.startsWith("Sp"));
        assertFalse(value.isEmpty());
    }

    @Test
    void shouldThrowArithmeticException() {
        assertThrows(ArithmeticException.class, () -> {
            int result = 10 / 0;
        });
    }
}

```

Розроб UserServiceTest

```

package org.example.service;

import org.example.model.dto.UserDto;
import org.example.model.entity.User;
import org.example.model.enums.UserRole;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

```



```

import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @InjectMocks
    private UserService userService;

    @Test
    void shouldCreateUser() {
        UserDto dto = UserDto.builder()
            .name("Иван Иванов")
            .email("ivan@example.com")
            .phone("+79990001122")
            .deviceToken("device-token-123")
    }

```

```

        .telegramChatId("123456789")
        .role(UserRole.ROLE_USER)
        .build();

        User savedUser = new User();
        savedUser.setId(1L);
        savedUser.setName(dto.getName());
        savedUser.setEmail(dto.getEmail());
        savedUser.setPhone(dto.getPhone());
        savedUser.setDeviceToken(dto.getDeviceToken());
        savedUser.setTelegramChatId(dto.getTelegramChatId());
        savedUser.setRole(UserRole.ROLE_USER);
        savedUser.setPassword("encoded-default-password");

        when(passwordEncoder.encode("default123")).thenReturn("encoded-default-
password");

        when(userRepository.save(any(User.class))).thenReturn(savedUser);

        User result = userService.createUser(dto);

        assertNotNull(result);
        assertEquals(1L, result.getId());
        assertEquals("Иван Иванов", result.getName());
        assertEquals("ivan@example.com", result.getEmail());
        assertEquals(UserRole.ROLE_USER, result.getRole());
        verify(userRepository, times(1)).save(any(User.class));
    }

```

```

                                                    @Test
void shouldReturnAllUsers() {
    User user = new User();
        user.setId(1L);
        user.setName("Иван");
        user.setEmail("ivan@example.com");
        user.setRole(UserRole.ROLE_USER);

    when(userRepository.findAll()).thenReturn(List.of(user));

    List<User> result = userService.getAllUsers();

        assertEquals(1, result.size());
    assertEquals("Иван", result.get(0).getName());
        verify(userRepository, times(1)).findAll();
}

```

```

                                                    @Test
void shouldGetUserById() {
    User user = new User();
        user.setId(1L);
        user.setEmail("ivan@example.com");

    when(userRepository.findById(1L)).thenReturn(Optional.of(user));

    User result = userService.getUserById(1L);

        assertNotNull(result);
}

```

```

        assertEquals(1L, result.getId());
        assertEquals("ivan@example.com", result.getEmail());
    }

    @Test
    void shouldThrowExceptionWhenUserNotFound() {
        when(userRepository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(IllegalArgumentException.class, () ->
            userService.getUserById(99L));
    }

    @Test
    void shouldUpdateUserAndRole() {
        User existingUser = new User();
        existingUser.setId(1L);
        existingUser.setName("Старое имя");
        existingUser.setEmail("old@example.com");
        existingUser.setRole(UserRole.ROLE_USER);

        UserDto dto = UserDto.builder()
            .name("Новое имя")
            .email("new@example.com")
            .phone("+79991112233")
            .deviceToken("new-device")
            .telegramChatId("555")
            .role(UserRole.ROLE_ADMIN)
            .build();
    }

```

```

        when(userRepository.findById(1L)).thenReturn(Optional.of(existingUser));
        when(userRepository.save(existingUser)).thenReturn(existingUser);

        User result = userService.updateUser(1L, dto);

        assertEquals("Новое имя", result.getName());
        assertEquals("new@example.com", result.getEmail());
        assertEquals(UserRole.ROLE_ADMIN, result.getRole());
        verify(userRepository, times(1)).save(existingUser);
    }

    @Test
    void shouldDeleteUser() {
        User user = new User();
        user.setId(1L);

        when(userRepository.findById(1L)).thenReturn(Optional.of(user));

        userService.deleteUser(1L);

        verify(userRepository, times(1)).delete(user);
    }
}

```

Разбор UserServiceVerifyTest

Файл отсутствует в архиве.

Разбор NotificationServiceTest

```
package                                     org.example.service;

import                                     org.example.model.dto.NotificationDto;
import                                     org.example.model.entity.Notification;
import                                     org.example.model.entity.User;
import                                     org.example.model.enums.NotificationChannel;
import                                     org.example.model.enums.NotificationStatus;
import                                     org.example.repository.NotificationRepository;
import                                     org.example.repository.UserRepository;
import                                     org.junit.jupiter.api.Test;
import                                     org.junit.jupiter.api.extension.ExtendWith;
import                                     org.mockito.InjectMocks;
import                                     org.mockito.Mock;
import                                     org.mockito.junit.jupiter.MockitoExtension;

import                                     java.util.List;
import                                     java.util.Optional;

import                                     static org.junit.jupiter.api.Assertions.*;
import                                     static org.mockito.ArgumentMatchers.any;
import                                     static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {

                                     @Mock
                                     private NotificationRepository notificationRepository;
```

```

                                                    @Mock
        private UserRepository userRepository;

                                                    @InjectMocks
        private NotificationService notificationService;

                                                    @Test
        void shouldCreateNotification() {
            User user = new User();
            user.setId(1L);
            user.setEmail("ivan@example.com");

            NotificationDto dto = NotificationDto.builder()
                .title("Напоминание")
                .message("Завтра пара по Spring")
                .channel(NotificationChannel.EMAIL)
                .recipientId(1L)
                .build();

            Notification savedNotification = new Notification();
            savedNotification.setId(10L);
            savedNotification.setTitle(dto.getTitle());
            savedNotification.setMessage(dto.getMessage());
            savedNotification.setChannel(dto.getChannel());
            savedNotification.setStatus(NotificationStatus.CREATED);
            savedNotification.setRecipient(user);

```

```

        when(userRepository.findById(1L)).thenReturn(Optional.of(user));
        when(notificationRepository.save(any(Notification.class))).thenReturn(savedNotification);
    }

```

```

    Notification result = notificationService.createNotification(dto);

    assertNotNull(result);
    assertEquals(10L, result.getId());
    assertEquals("Напоминание", result.getTitle());
    assertEquals(NotificationChannel.EMAIL, result.getChannel());
    assertEquals(NotificationStatus.CREATED, result.getStatus());
    assertEquals(user, result.getRecipient());
    verify(userRepository, times(1)).findById(1L);
    verify(notificationRepository, times(1)).save(any(Notification.class));
}

```

```

@Test
void shouldThrowExceptionWhenRecipientNotFound() {
    NotificationDto dto = NotificationDto.builder()
        .title("Напоминание")
        .message("Сообщение")
        .channel(NotificationChannel.EMAIL)
        .recipientId(99L)
        .build();

    when(userRepository.findById(99L)).thenReturn(Optional.empty());

    assertThrows(IllegalArgumentException.class, () ->

```



```

notificationService.createNotification(dto));
        verify(notificationRepository, never()).save(any(Notification.class));
    }

    @Test
    void shouldGetNotificationById() {
        Notification notification = new Notification();
        notification.setId(1L);
        notification.setTitle("Tectr");

        when(notificationRepository.findById(1L)).thenReturn(Optional.of(notification));

        Notification result = notificationService.getNotificationById(1L);

        assertNotNull(result);
        assertEquals(1L, result.getId());
        assertEquals("Tectr", result.getTitle());
    }

    @Test
    void shouldThrowExceptionWhenNotificationNotFound() {
        when(notificationRepository.findById(404L)).thenReturn(Optional.empty());

        assertThrows(IllegalArgumentException.class, () ->
notificationService.getNotificationById(404L));
    }

```

```

                                                                    @Test
void shouldReturnNotificationsByStatus() {
    Notification notification = new Notification();
    notification.setStatus(NotificationStatus.CREATED);

    when(notificationRepository.findByStatus(NotificationStatus.CREATED)).thenReturn(List.of(notification));

    List<Notification> result =
notificationService.getNotificationsByStatus(NotificationStatus.CREATED);

    assertEquals(1, result.size());
    assertEquals(NotificationStatus.CREATED, result.get(0).getStatus());
    verify(notificationRepository,
times(1)).findByStatus(NotificationStatus.CREATED);
}

```

```

                                                                    @Test
void shouldDeleteNotification() {
    Notification notification = new Notification();
    notification.setId(1L);

    when(notificationRepository.findById(1L)).thenReturn(Optional.of(notification));

    notificationService.deleteNotification(1L);

    verify(notificationRepository, times(1)).delete(notification);
}

```

```
}  
  
}
```

Разбор NotificationServiceExceptionTest

Файл отсутствует в архиве.

Разбор UserControllerTest

```
package                                     org.example.controller;  
  
import                                     org.example.model.entity.User;  
import                                     org.example.model.enums.UserRole;  
import                                     org.example.service.UserService;  
import                                     org.junit.jupiter.api.BeforeEach;  
import                                     org.junit.jupiter.api.Test;  
import                                     org.mockito.Mockito;  
import                                     org.springframework.test.web.servlet.MockMvc;  
import                                     org.springframework.test.web.servlet.setup.MockMvcBuilders;  
  
import                                     java.util.List;  
  
import                                     static org.mockito.Mockito.when;  
import                                     static  
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;  
import                                     static  
org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;  
import                                     static  
org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;  
import                                     static
```

```
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```
class UserControllerTest {

    private MockMvc mockMvc;
    private UserService userService;

    @BeforeEach
    void setUp() {
        userService = Mockito.mock(UserService.class);
        UserController userController = new UserController(userService);
        mockMvc = MockMvcBuilders.standaloneSetup(userController).build();
    }

    @Test
    void shouldReturnAllUsers() throws Exception {
        User user = new User();
        user.setId(1L);
        user.setName("Иван Иванов");
        user.setEmail("ivan@example.com");
        user.setRole(UserRole.ROLE_USER);

        when(userService.getAllUsers()).thenReturn(List.of(user));

        mockMvc.perform(get("/users/all"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].id").value(1))
            .andExpect(jsonPath("$[0].name").value("Иван Иванов"))
    }
}
```

```

        .andExpect(jsonPath("$.email").value("ivan@example.com"))
        .andExpect(jsonPath("$.role").value("ROLE_USER"));
    }

    @Test
    void shouldReturnDebugUsersAsText() throws Exception {
        User user = new User();
        user.setId(1L);
        user.setName("Иван Иванов");
        user.setEmail("ivan@example.com");
        user.setRole(UserRole.ROLE_USER);

        when(userService.getAllUsers()).thenReturn(List.of(user));

        mockMvc.perform(get("/users/debug"))
            .andExpect(status().isOk())
            .andExpect(content().string(org.hamcrest.Matchers.containsString("COUNT = 1")))
            .andExpect(content().string(org.hamcrest.Matchers.containsString("ivan@example.com"))));
    }
}

```

Mockito и mock-объекты

Основные

возможности

Mockito:

@Mock	-	создание	mock-зависимости.
@InjectMocks	-	внедрение	mock-объектов.
when(...).thenReturn(...)	-	настройка поведения	mock.
verify(...)	-	проверка вызова	метода.
assertThrows(...)	-	проверка	исключений.

Mock-объекты позволяют тестировать сервисы без реальной базы данных.

Запуск тестов

Для запуска тестов использовались команды:

```
mvn test
mvn clean test
```

Тесты также можно запускать напрямую через IntelliJ IDEA.

Ответы на контрольные вопросы

Что такое unit-тест? Тест отдельного класса или метода.

Что делает @Test? Помечает метод как тестовый.

Что такое mock-объект? Заглушка, имитирующая зависимость.

Что делает Mockito? Создаёт mock-объекты и проверяет вызовы.

Что делает verify()? Проверяет вызов метода.

Что делает @WebMvcTest? Поднимает только web-слой Spring Boot.

Что такое MockMvc? Инструмент тестирования HTTP-запросов.

Что делает `assertThrows()`? Проверяет выброс исключения.

Вывод

В ходе лабораторной работы были изучены основы тестирования Spring Boot-приложений.

Были			реализованы:
-			unit-тесты;
-			web-тесты;
-			mock-объекты;
-	проверка		исключений;
-	тестирование	сервисного	слоя;
-	тестирование		контроллеров.

Также были изучены возможности JUnit 5 и Mockito.

Дополнительные разделы по методическим указаниям

JUnit

JUnit является основным фреймворком тестирования в Java. Он предоставляет аннотации, assertions и жизненный цикл тестов.

Mockito

Mockito используется для создания mock-объектов и изоляции зависимостей.

Mock и Stub

Mock - объект-заглушка. Stub - заранее заданное поведение mock-объекта.

@Mock и @InjectMocks

@Mock создает mock-объект. @InjectMocks внедряет mock-зависимости в тестируемый класс.

verify()

verify() проверяет, что нужный метод действительно был вызван.

WebMvcTest

@WebMvcTest используется для тестирования HTTP-слоя и контроллеров.

assertThrows

assertThrows проверяет выбрасывание ожидаемого исключения.

Unit и Integration тесты

Unit-тесты проверяют отдельный класс. Integration-тесты проверяют совместную работу компонентов.

Дополнительные ответы на контрольные вопросы

Что такое unit-тест? Тест отдельного класса или метода.

Что такое mock? Объект-заглушка для имитации зависимости.

Что делает when().thenReturn()? Описывает поведение mock-объекта.

Что делает `verify()`? Проверяет вызов метода.

Что делает `@Test`? Помечает тестовый метод.

Что делает `@BeforeEach`? Выполняется перед каждым тестом.

Что делает `assertEquals()`? Сравнивает ожидаемое и фактическое значение.

Зачем нужны тесты? Для проверки корректности работы приложения.